

PRACTICAL WEEK-7

TASK1

Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 100;    // Global variable
static int static_var = 200; // Static variable

void code_function()
{
    printf("This is a function.\n");
}

int main()
{
    int stack_var = 300;    // Stack variable
    int *heap_var;          // Pointer variable on stack

    heap_var = (int *)malloc(sizeof(int));
    *heap_var = 400;

    printf("Linux Process Address Space\n");
    printf("=====\n\n");

    printf("Address of code variable/function : %p\n", (void *)code_function);
    printf("Address of global variable       : %p\n", (void *)&global_var);
    printf("Address of static variable        : %p\n", (void *)&static_var);
    printf("Address of heap variable          : %p\n", (void *)heap_var);
    printf("Address of stack variable         : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

```
laasya@laasya:~$ ls
client          practical      signals.c      week4p_task2
client.c        practical_week2 skill_week4    week4p_task2.c
command_executor seesion5      task5          wwkek4p_task1
command_executor.c server         task5.c        wwkek4p_task1.c
information     server.c      'tmp information' wwkek4p_task2
information.txt server_fifo    week2          wwkek4p_task2.c
memoery.c       session6      week3
ossp            signals       week4
laasya@laasya:~$ gcc memory.c -o memory
```

TASK-2

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;
static int static_var = 200;

int main() {
    int stack_var = 300;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 400;

    printf("Process ID (PID): %d\n", getpid());

    printf("Address of global variable : %p\n", (void *)&global_var);
    printf("Address of static variable : %p\n", (void *)&static_var);
    printf("Address of stack variable : %p\n", (void *)&stack_var);
    printf("Address of heap variable : %p\n", (void *)heap_var);
    printf("Address of main function : %p\n", (void *)main);

    printf("\nProcess is running...\n");
    printf("Press Ctrl+C to terminate.\n");

    while (1) {
        sleep(1);
    }

    free(heap_var);

    return 0;
}

```

```

laasya@laasya: ~
laasya@laasya:~$ nano memory.c
laasya@laasya:~$ gcc -o memory memory.c
laasya@laasya:~$ ls
client          ossp            signals.c       week4p_task2.c
client.c        practical       skill_week4     week8
command_executor practical_week2 task5            wwek4p_task1
command_executor.c seesion5       task5.c         wwek4p_task1.c
information     server         'tmp information' wwek4p_task2
information.txt server.c        week2           wwek4p_task2.c
memoery.c       server_fifo    week3
memory          session6      week4
memory.c        signals       week4p_task2
laasya@laasya:~$ |

```

memory.c

signals

week4p_task2

laasya@laasya:~\$./memory

Process ID (PID): 2189

Address of global variable : 0x6496eeada010

Address of static variable : 0x6496eeada014

Address of stack variable : 0x7ffc0e98c63c

Address of heap variable : 0x64970d7fb2a0

Address of main function : 0x6496eead71c9

Process is running...

Press Ctrl+C to terminate.

|